In [1]:
```
import pandas as pd
```

In [4]:
```
movies=pd.read_csv(r'D:\Daily assignments\3rd Feb Kaggle\archive\movie.csv')
```

In [5]:
```
movies.shape
```

Out[5]: (27278, 3)

In [6]:
```
movies.head(1)
```

Out[6]:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy

In [8]:
```
ratings=pd.read_csv(r'D:\Daily assignments\3rd Feb Kaggle\archive\rating.csv')
```

In [10]:
```
ratings.shape
```

Out[10]: (20000263, 4)

In [11]:
```
ratings.head(1)
```

Out[11]:

	userId	movieId	rating	timestamp
0	1	2	3.5	2005-04-02 23:53:47

In [12]:
```
tags=pd.read_csv(r'D:\Daily assignments\3rd Feb Kaggle\archive\tag.csv')
```

In [13]:
```
tags.shape
```

Out[13]: (465564, 4)

In [14]:
```
tags.head(1)
```

Out[14]:

	userId	movieId	tag	timestamp
0	18	4141	Mark Waters	2009-04-24 18:19:40

In [15]:
```
print(movies.shape)
print(ratings.shape)

print(tags.shape)
```

```
(27278, 3)
(20000263, 4)
(465564, 4)
```

In [18]:
```
print(movies.columns)
print(ratings.columns)
print(tags.columns)
```

```
Index(['movieId', 'title', 'genres'], dtype='object')
Index(['userId', 'movieId', 'rating', 'timestamp'], dtype='object')
Index(['userId', 'movieId', 'tag', 'timestamp'], dtype='object')
```

In [19]:
```
del ratings['timestamp'] #timestamp deleted
del tags['timestamp']
```

In [20]:
```
print(movies.columns)
print(ratings.columns)
print(tags.columns)
```

```
Index(['movieId', 'title', 'genres'], dtype='object')
Index(['userId', 'movieId', 'rating'], dtype='object')
Index(['userId', 'movieId', 'tag'], dtype='object')
```

In [22]:
```
tags.head(1)
```

Out[22]:

	userId	movieId	tag
0	18	4141	Mark Waters

In [23]:
```
tags.head(5)
```

Out[23]:

	userId	movieId	tag
0	18	4141	Mark Waters
1	65	208	dark hero
2	65	353	dark hero
3	65	521	noir thriller
4	65	592	dark hero

In [25]:
```
row_0=
tags.iloc[1] #Series Data Structures
```

Out[25]:
```
userId            65
movieId          208
tag        dark hero
Name: 1, dtype: object
```

In [ ]:
```
print(tags[0]
```

In [26]:
```
row_0 = tags.iloc[1]
```

In [27]:
```
print(row_0)
```

```
userId            65
movieId          208
tag         dark hero
Name: 1, dtype: object
```

In [28]:
```
row_0.index
```

Out[28]:
```
Index(['userId', 'movieId', 'tag'], dtype='object')
```

In [31]:
```
row_0['userId']
```

Out[31]:
```
np.int64(65)
```

In [32]:
```
'rating' in row_0
```

Out[32]:
```
False
```

In [33]:
```
row_0.name
```

Out[33]:
```
1
```

In [34]:
```
row_0 = row_0.rename('firstRow')
row_0.name
```

Out[34]:
```
'firstRow'
```

DataFrames¶

In [35]:
```
tags.head()
```

Out[35]:

	userId	movieId	tag
0	18	4141	Mark Waters
1	65	208	dark hero
2	65	353	dark hero
3	65	521	noir thriller
4	65	592	dark hero

In [36]:
```
tags.index
```

Out[36]:
```
RangeIndex(start=0, stop=465564, step=1)
```

In [37]:
```
tags.columns
```

Out[37]:
```
Index(['userId', 'movieId', 'tag'], dtype='object')
```

In [38]:
```
tags.iloc[ [0,11,500] ]
```

Out[38]:

	userId	movieId	tag
0	18	4141	Mark Waters
11	65	1783	noir thriller
500	342	55908	entirely dialogue

Descriptive Statistics

In [39]:
```
ratings['rating'].describe()
```

Out[39]:
```
count    2.000026e+07
mean     3.525529e+00
std      1.051989e+00
min      5.000000e-01
25%      3.000000e+00
50%      3.500000e+00
75%      4.000000e+00
max      5.000000e+00
Name: rating, dtype: float64
```

In [40]:
```
ratings.describe()
```

Out[40]:

	userId	movieId	rating
count	2.000026e+07	2.000026e+07	2.000026e+07
mean	6.904587e+04	9.041567e+03	3.525529e+00
std	4.003863e+04	1.978948e+04	1.051989e+00
min	1.000000e+00	1.000000e+00	5.000000e-01
25%	3.439500e+04	9.020000e+02	3.000000e+00
50%	6.914100e+04	2.167000e+03	3.500000e+00
75%	1.036370e+05	4.770000e+03	4.000000e+00
max	1.384930e+05	1.312620e+05	5.000000e+00

In [41]:
```
ratings['rating'].mean()
```

Out[41]:
```
np.float64(3.5255285642993797)
```

In [42]:
```
ratings.mean()
```

Out[42]:
```
userId     69045.872583
movieId     9041.567330
rating         3.525529
dtype: float64
```

In [43]:
```
ratings['rating'].max()
```

Out[43]:
```
5.0
```

In [44]:
```
ratings['rating'].std()
```

Out[44]:
```
1.051988919275684
```

In [45]:
```
ratings['rating'].mode()
```

Out[45]:
```
0    4.0
Name: rating, dtype: float64
```

In [46]:
```
ratings.corr()
```

Out[46]:

	userId	movieId	rating
userId	1.000000	-0.000850	0.001175
movieId	-0.000850	1.000000	0.002606
rating	0.001175	0.002606	1.000000

In [47]:
```
filter1 = ratings['rating'] > 10
print(filter1)
filter1.any()
```

```
0           False
1           False
2           False
3           False
4           False
            ...
20000258    False
20000259    False
20000260    False
20000261    False
20000262    False
Name: rating, Length: 20000263, dtype: bool
```

Out[47]:
```
np.False_
```

In [48]:
```
filter2 = ratings['rating'] > 0
filter2.all()
```

Out[48]:
```
np.True_
```

Data Cleaning: Handling Missing Data

In [49]:
```
movies.shape
```

Out[49]:
```
(27278, 3)
```

In [50]:
```
movies.isnull().any().any()
```

Out[50]:
```
np.False_
```

In [51]:
```
ratings.shape
```

Out[51]:
```
(20000263, 3)
```

In [52]:
```
ratings.isnull().any().any()
```

Out[52]:
```
np.False_
```

In [53]:
```
tags.shape
```

Out[53]:
```
(465564, 3)
```

In [54]:
```
tags.isnull().any().any()
```

Out[54]:
```
np.True_
```

In [55]:
```
tags=tags.dropna()
```

In [56]:
```
tags.isnull().any().any()
```

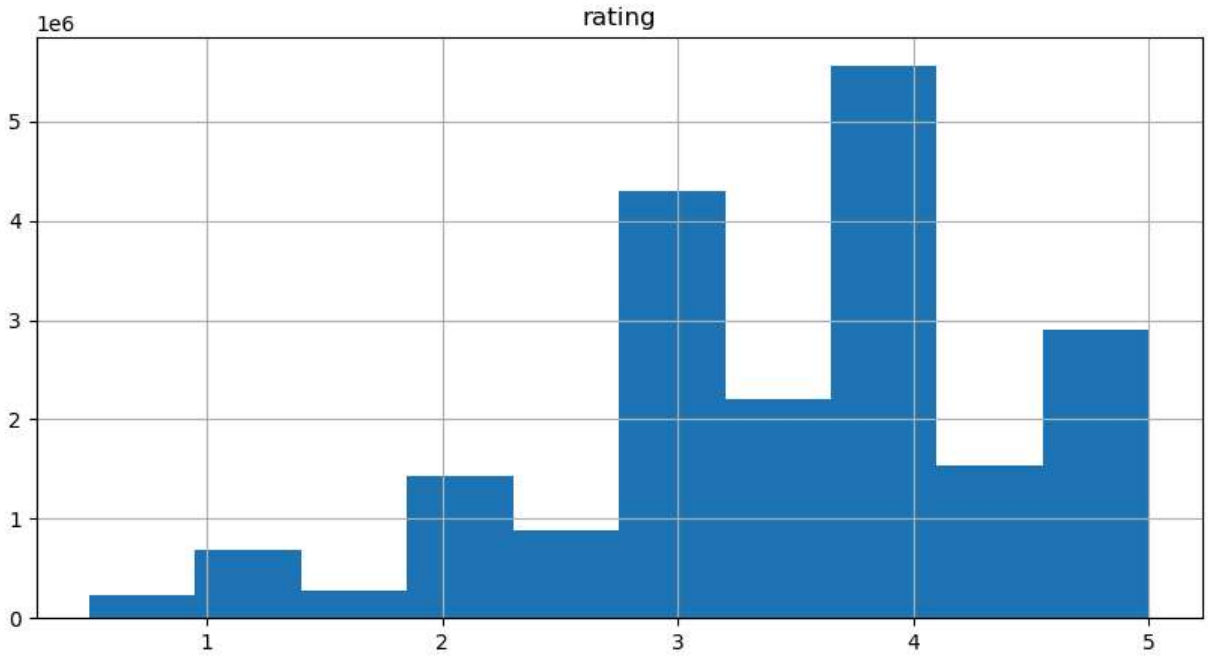
Out[56]:
```
np.False_
```

In [57]:
```
tags.shape
```

Out[57]:
```
(465548, 3)
```

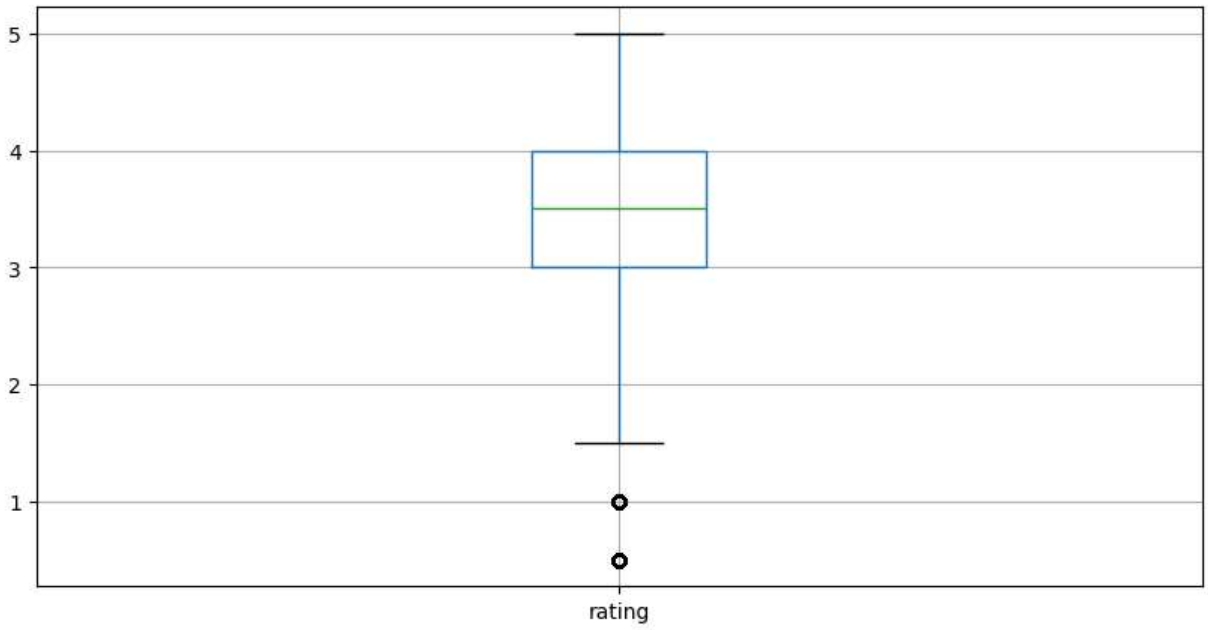
In [58]:
```
%matplotlib inline

ratings.hist(column='rating', figsize=(10,5))
```

Out[58]:
```
array([[<Axes: title={'center': 'rating'}>]], dtype=object)
```



In [62]:
```
ratings.boxplot(column='rating', figsize=(10,5))
```

Out[62]: <Axes: >



Slicing Out Columns

In [63]:
```
tags['tag'].head()
```

Out[63]:
```
0      Mark Waters
1        dark hero
2        dark hero
3    noir thriller
4        dark hero
Name: tag, dtype: object
```

In [64]:
```
movies[['title','genres']].head()
```

Out[64]:

	title	genres
0	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	Jumanji (1995)	Adventure|Children|Fantasy
2	Grumpier Old Men (1995)	Comedy|Romance
3	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	Father of the Bride Part II (1995)	Comedy

In [65]:
```
ratings[-10:]
```

Out[65]:

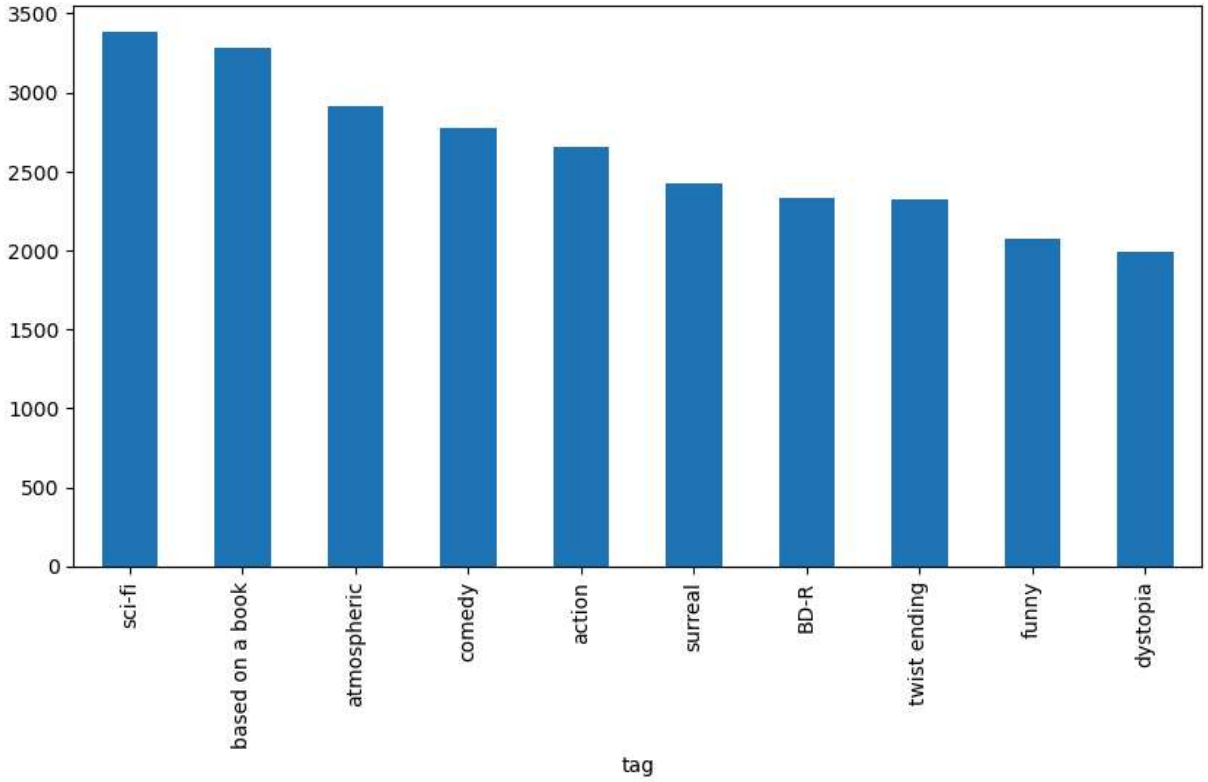
	userId	movieId	rating
20000253	138493	60816	4.5
20000254	138493	61160	4.0
20000255	138493	65682	4.5
20000256	138493	66762	4.5
20000257	138493	68319	4.5
20000258	138493	68954	4.5
20000259	138493	69526	4.5
20000260	138493	69644	3.0
20000261	138493	70286	5.0
20000262	138493	71619	2.5

In [66]:
```
tag_counts = tags['tag'].value_counts()
tag_counts[-10:]
```

Out[66]:
```
tag
missing child                    1
Ron Moore                        1
Citizen Kane                     1
mullet                           1
biker gang                       1
Paul Adelstein                   1
the wig                          1
killer fish                      1
genetically modified monsters    1
topless scene                    1
Name: count, dtype: int64
```

In [67]:
```
tag_counts[:10].plot(kind='bar', figsize=(10,5))
```

Out[67]:
```
<Axes: xlabel='tag'>
```



In [ ]:
```
```